

TP3**Signaux aléatoires**

Le but de ce TP est de se familiariser avec quelques fonctions Matlab permettant de manipuler des processus aléatoires.

1. Réalisation d'un bruit blanc gaussien

Générer N échantillons d'un bruit blanc gaussien de moyenne nulle et de variance 1 (utiliser la fonction "`randn(1, N)`"). On prendra $N=128, 256, 512$ et 1024 . Calculer alors la moyenne (fonction "`mean`") et la variance (fonction "`var`") pour chaque réalisation. Faites plusieurs essais et vérifiez que la moyenne et la variance renvoyées ne sont pas les valeurs escomptées, mais des approximations de ces valeurs.

Pour corriger ce problème, on va successivement normaliser, blanchir et centrer le signal afin d'obtenir un « vrai » bruit blanc.

Pour cela, on va appliquer le code Matlab suivant :

```
e = randn (1,N) ;  
E = fft (e) ;  
Enorm = E./abs(E) ; % Normalisation  
bb = sqrt(N)*ifft(Enorm)*variance; % Blanchiment  
bb = real(bb);  
bbc = bb -mean(bb); % Centrage
```

Travail à faire :

- 1.1 Ecrire une fonction Matlab qui, connaissant la variance et le nombre d'échantillons N , génère un `bbc` (bruit blanc centré).
- 1.2 En exécutant plusieurs fois cette fonction, vérifier que les `bbc` générés sont centrés et de variance égale à celle choisie.
- 1.3 Comment peut-on obtenir à partir du `bbc` généré un bruit blanc de moyenne différente de 0.
- 1.4 Générer 10000 échantillons d'un bruit blanc centré et de variance égale à 2.

Utiliser la commande Matlab : `[yvalues,xvalues]=hist(x,20);` pour générer les abscisses (xvalues) et les ordonnées (yvalues) de l'histogramme de 20 barres représentant les 10000 échantillons.

Tracer alors l'histogramme normalisé à l'aide de la commande `bar(xvalues,yvalues/N);`

1.5 La densité de probabilité d'un processus gaussien est donnée par la relation :

$$p(x) = \frac{1}{\sigma\sqrt{2\pi}} e^{-\frac{(x-\mu)^2}{2\sigma^2}}$$

où μ est la moyenne du processus et σ^2 sa variance. Générer et tracer cette densité sur le même histogramme obtenu ci-dessus.

2. La fonction « random »

Dans Matlab, la fonction « random » peut être utilisée pour générer des vecteurs ou matrices aléatoires. Sa syntaxe est la suivante :

R=random('la Loi de probabilité', Paramètre1, ..., ParamètreN, nombre de ligne, nombre de colonnes)

La loi de probabilité peut être par exemple 'norm' pour la loi de probabilité normale ou gaussienne, 'unif' pour la loi uniforme, 'unid' pour la loi uniforme discrète, Tapez **help random** pour voir toutes les possibilités.

Par exemple, la commande :

`R=random('norm', 3, sqrt(2), 1, 20);`

génère un vecteur ligne contenant 20 éléments aléatoires suivant une distribution gaussienne de moyenne 3 et de variance 2.

Pour générer un entier aléatoire uniformément distribué entre 1 et 31, on utilise la commande :

`R=random('unid', 31, 1, 1);`

ou tout simplement :

`R=random('unid', 31);`

Travail à faire :

Dans chaque cas, exécuter chaque programme plusieurs fois pour voir la nature aléatoire des réalisations.

2.1 Amplitude aléatoire

Considérons le signal :

$$x(t) = A \cos(2\pi 4t)$$

Effectuer 4 réalisations du signal ci-dessus en considérant que l'amplitude A est un entier aléatoire uniformément distribué entre $[1, 4]$. Considérer 1024 échantillons entre 0 et 1.

Tracer dans une même figure les 4 réalisations obtenues (utiliser la commande subplot).

2.2 Phase aléatoire

Considérons le signal :

$$x(t) = \cos(2\pi 4t + \Phi)$$

Faites le même travail que ci-dessus en considérant maintenant que la phase Φ est uniformément distribuée entre $[-\pi, \pi]$.

2.3 Fréquence aléatoire

Considérons le signal :

$$x(t) = \cos(2\pi ft)$$

Faites le même travail en considérant que la fréquence f est un entier uniformément distribuée entre $[1, 4]$.

2.4 Sinusoïde avec amplitude, fréquence et phase

Considérons le signal :

$$x(t) = A \cos(2\pi ft + \Phi)$$

Faites le même travail que ci-dessus en considérant que :

- ☞ A est un entier aléatoire uniformément distribué entre $[1, 4]$,
- ☞ f est un entier uniformément distribuée entre 1 et 4,
- ☞ Φ est uniformément distribuée entre $[-\pi, \pi]$.